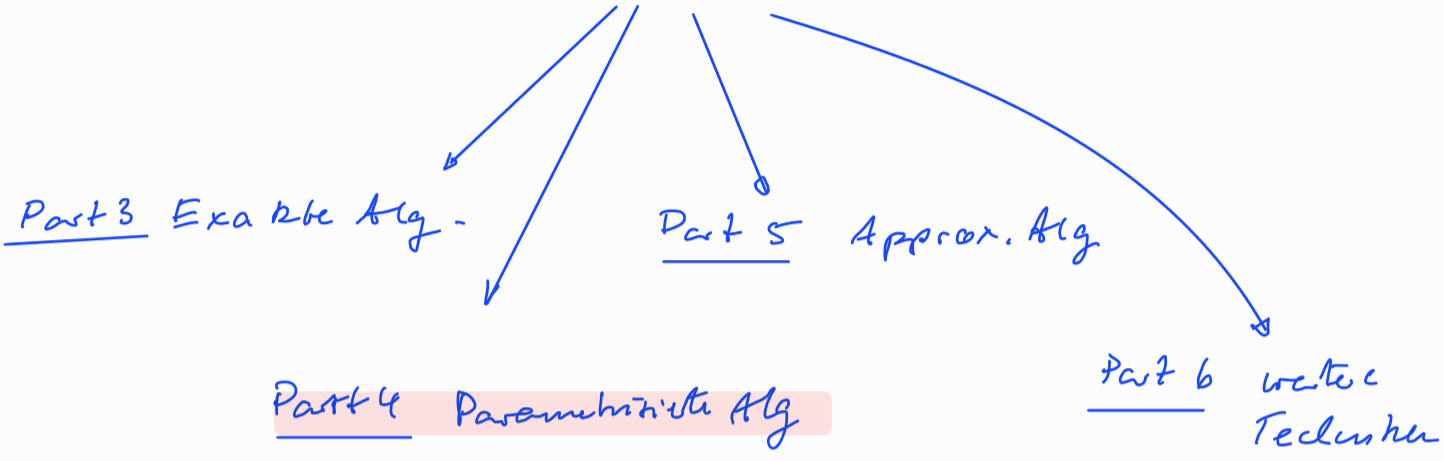
Fahrplan

Part 1 TM, Algorithmen & Entscheidbarkeit

Part 2 Einfach vs schwere Probleme & NP vollständigkeit



Part 3 Exakte Alg.

Part 4 Parameterisierte Alg

Part 5 Approx. Alg

Part 6 weitere Techniken

4. Parametrisierte Alg.

Das Ziel ist es Parameter zu detektieren, in denen die exponentielle Laufzeit für exakte Algorithmen steckt.

Ein Algorithmus mit Laufzeit $f(k) \cdot n^c$ für Konstante c die unabhängig von k & n ist, heißen fixed-parameter tractable (FPT)-algorithmus.

Wir haben schon ein Bsp gesehen für Vertex Cover wo wir einen Alg. mit Laufzeit $O(2^k \text{ poly}(n))$ gefunden haben

Def: Ein **parametrisiertes Problem** ist eine Sprache $L \subseteq \Sigma^* \times \mathbb{N}$, Σ endl. Alphabet.
Für eine instance $I = (x, k) \in \Sigma^* \times \mathbb{N}$ heißt k der parameter

Ein parametrisiertes Problem L ist **fixed-parameter tractable (FPT)**, wenn es

- einen FPT-Algorithmus A gibt &
- berechenbare Fkt $f: \mathbb{N} \to \mathbb{N}$ &
  [∃ TM M mit input $k \in \mathbb{N}$ & output $f(k)$, d.h. M hält]
- Konstante c

so dass für gegebenes $I = (x, k) \in \Sigma^* \times \mathbb{N}$, der Alg. A in $O(f(k) \cdot |I|^c)$ entscheidet ob $(x, k) \in L$ oder nicht.

↑ kann man sich sparen, da man Konstante in f schieben kann

Die Komplexitätsklasse aller fixed-parameter-tractable probleme heißt **FPT**.

Im wesentlichen ist das Äquivalent zu:

D ist Entscheidungsproblem

& parameter $p: I \to p(I)$ als Abbildung die jeder Instanz I von D den parameter $p(I)$ zuweist.

Dann ist (D, p) das parameterisierte Problem.

& $(D, p) \in FPT$ wenn es $O(|I|^c \cdot f(p(I))$-Zeitalgorithmus zum lösen des Probl. gibt.

Bsp: D Vertex Cover problem.

Instanz $I = (b, k)$

im Bsp $p(I) = k$ (max erlaubte größe VC)

es wäre aber auch $p(I) = |V(b)|$ möglich

oder $p(I) = \max_{v \in V} |N(v)|$ = max-degree

Im Folgenden betrachten wir Techniken für das Design von FPT-algorithmen.

- Kernelization
- Beschränkte Suchbäume

(° Dyn. Progr. auf strukturr. Parametern.)

Grenzen von FPT

4.1 Kernelisierung

Ziel ist pre-processing der Input instanz
& diese durch strukturelle Eigenschaften (z.B. sparse, zuerst einfache Fälle abgrasen)
auf einen Kern zu verkleinern.
so dass "Kern" Ja-antwort hat $\Leftrightarrow$ ursprüngliche Instanz Ja-antwort hat.

Auf dem kleinen Kern kann man dann auch langsame Algorithmen anwenden.

Def: Eine **Kernelisierungsalgorithmus** für parametrisiertes Problem (D, p) ist poly-zeit Transformation (reduktion) von Instanz $(I, p(I)) \in (D, p)$ auf Instanz $(I', p(I')) \in (D, p)$

so dass
- I Ja-instanz von D $\Leftrightarrow$ I' Ja-inst von D
- es exist. berechenbare Fkt $g: \mathbb{N} \to \mathbb{N}$ die nur von $p(I)$ abhängt & $|I'| \leq g(p(I))$ erfüllt

(die größe von I' hängt nur noch vom parameter $p(I)$ ab, nicht von Input-größe $|I|$)

In diesem Fall heißt I' **Kernel** von I & $g(p(I))$ **größe von Kernel**

Schauen wir uns jetzt konkrete Bsp an.

Vertex-Cover Problem (VCP)

input: undirected graph $G = (V, E)$, integer k

question: Does G have a vertex cover of size $\leq k$, i.e.,
is there a subset $C \subseteq V$
s.t: (i) $|C| \leq k$
(ii) $C \cap e \neq \emptyset \quad \forall e \in E$

Verschiedene strukturelle Eigenschaften von G können ausgenutzt werden um Input-Instanz (G, k) zu einer "kleineren" Instanz (G', k') zu transformieren.

Buss-Reduktion

Beobachtung 1: Wenn G Knoten v enthält mit $\deg(v) = 0$, dann ist v in keiner Menge von G enthalten & dementsprechend ist v vollkommen vernachlässigbar für alle vertex covers von G.

$\Rightarrow$ **Regel 1:** (G, k) ist JA-Instanz
$\Leftrightarrow$ $(G - v, k)$ ist JA-Instanz $\forall v \in V$ mit $\deg(v) = 0$.

Beobachtung 2: Wenn G Knoten v mit $\deg(v) \geq k+1$ dann muss v in jedem VC von G sein denn wenn v nicht enthalten ist in VC C dann müssen alle Nachbarn von v drin sein $\Rightarrow |C| \geq k+1$ ↯.

$\Rightarrow$ **Regel 2:** (G, k) ist JA-Instanz $\Leftrightarrow$
$(G - v, k-1)$ ist JA-Instanz f.a. $v \in V$ mit $\deg(v) \geq k+1$

5

Wir wollen jetzt diese 2 Regeln um eine Kernelization von VC zu formulieren. Dazu betrachte folgenden Algorithmus.

VC_kernel (G, k)

Input: (G, k)
IF $(k < 0)$ return $I_{NEIN} = (\bullet\!\!-\!\!\bullet, 0)$
WHILE (in G gibt es Knoten v mit $\deg(v) > k$ OR $\deg(v) = 0$)
  Sei v ein solcher Knoten
  IF $(\deg(v) = 0)$ THEN $G \leftarrow G - v$
  ELSE $G \leftarrow G - v$
       $k \leftarrow k - 1$
Sei $(G' = (V', E'), k')$ die resultierende Instanz
IF ($|V'| > 2k^2$ OR $|E'| > k^2$) THEN
  Return $I_{NEIN} = (\bullet\!\!-\!\!\bullet, 0)$ (da sicher $g(k) = k' = 2$)
ELSE Return (G', k')

um zu zeigen, dass dies ein valider = Kernelization = Alg ist müssen wir zeigen dass
- Alg. in Polyzeit läuft
- I yes $\Leftrightarrow$ I' yes
- $|I'| \leq g(\gamma(I))$ für berechenbare Fkt g

Thm (Buss-Kernel)

VC_kernel (G, k) ist ein $O(|V||E|)$-Zeit Kernelisierungsalg.
Insbesondere hat VC einen Kernel der Größe $\leq k^2$,
da es gilt: die zurückgegebene Instanz $I' = (G', k')$ hat
$|I'| = |E(G')| + |V(G')| \leq 3k^2 = g(k)$

Beweis:

Polyzeit $O(|V||E|)$ (i^{te} Aufg) ✓

müssen jetzt zeigen: (G, k) JA-Instanz
$\Leftrightarrow$ returned (G', k') ist JA-Instanz.

Dafür erstmal folgende Behauptung:

Wenn (G,k) JA-Instanz,
dann gibt für letzte konstruierte
Instanz (G',k') [vor 2. "IF-Bed"]
dass $|E(G')| \leq k'^2$ & $|V(G')| \leq 2k^2$

($G = \dots \Rightarrow G' = \emptyset$! auch OK, dann trivial erfüllt)

Bew. Beh: wegen Konstr. gilt: $\forall u \in V(G'): \deg(u) \leq k'$

$\Rightarrow$ jeder Knoten ist in $\leq k'$ Kanten enthalten (*)

da (G,k) JA-Instanz, dann folgt aus
Argumentation für Regel 1 & 2, dass
(G',k') JA-Instanz ist

$\Rightarrow$ G' hat VC C' der größe $|C'| \leq k'$ (**)

$\Rightarrow$ $|E(G')| \leq k' \cdot k' = k'^2$ (*)? (**)

Jede Kante aus G' besteht aus
2 Knoten von $V(G')$.

$\Rightarrow$ $|V(G')| = \bigcup_{e \in E(G')} e \leq 2 \cdot k'^2$

($\dots$ max $|V(G')| = 2k'^2$)

✓ qed Beh.

$\Rightarrow$ wenn (G,k) JA-Instanz $\Rightarrow$ im Alg. wird
(G',k') zurückgegeben
(letzte "ELSE"-Bed).

kann eben (implizit) gezeigt werden auf (G', k')
keine der Regeln 1 & 2 anwendbar ist. Da gilt:

wenn (G', k') JA-Inst $\Rightarrow |V(G')| \leq 2k'^2$ & $|E(G')| \leq k'^2$

an kontrapositiven folgt: $|V(G')| > 2k'^2$ or $|E(G')| > k'^2$ $\Rightarrow (G', k')$ ist NEIN Inst.

d.h. in diesem Fall wird korrekt NEIN Instanz I_{NEIN} zurückgegeben.

für alle zurückgegebenen Instanzen (G', k') (auch für I_{NEIN}) gilt insbesondere

$$|I'| = |E(G')| + |V(G')| \leq 3k'^2 \leq \underbrace{3k^2}_{g(k)}$$

$\square$

$\Rightarrow$ für bel (G, k) erhalten wir kleine Instanz (G', k') mit $O(k^2)$ Kanten & Knoten.

Auf diesem kernel (G', k') kann man bel. exponentielle Alg ausführen, um festzustellen ob es zu (G', k') ein $VC\ C'$ mit $|C'| \leq k'$ gibt, d.h. ob (G', k') eine JA-Instanz ist $\Rightarrow (G, k)$ JA-Instanz & VC C von (G, k) kann man aus C' rekonstruieren

Aus letzten Betrachtungen ergibt sich folgender FPT-Alg:

VC_FPT (G, k)

- $(G', k') \leftarrow$ VC_Kernel. $\quad \mathcal{O}(|V||E|)$
- Brute-Force(G', k'): all $\binom{V'}{k}$ subsets $\quad \mathcal{O}(|E'| \cdot |V'|^k)$
  test ob V & return VC C' von G'
  $\leq \mathcal{O}(k^2 (2k^2)^k)$
- Füge entsprechende Knoten von G aus Ker. hinzu. $\Rightarrow C$ $\quad \mathcal{O}(|V|)$
- return C.

$|E'| \leq k'^2 \leq k^2$
$|V'| \leq 2k'^2 \leq 2k^2$

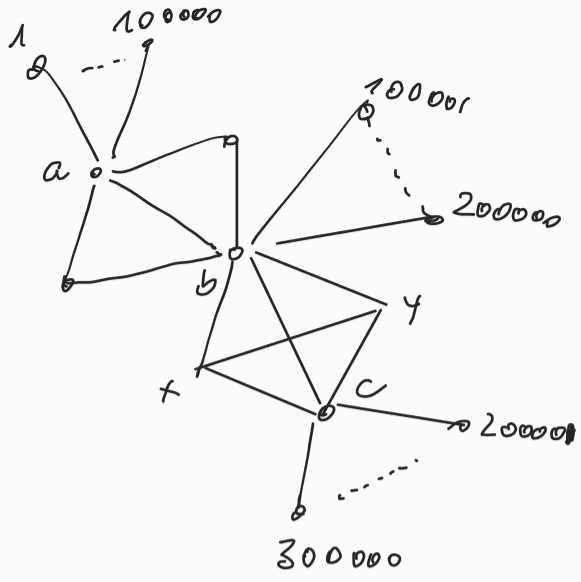
$\Rightarrow$ overall runtime: $\mathcal{O}(|V||E| + f(k))$
$\in \mathcal{O}(|V||E| \cdot \tilde{f}(k))$
mit $\tilde{f}(k) = f(k) + 1$

↗ wann? (nicht)

oBdA
$|V||E| \geq 1$ $\Rightarrow$ $f(k) \leq f(k) \cdot |I|$
$\|$
$|I|$ $\qquad |I| \leq 1$

$\Rightarrow |I| + f(k) \leq (f(k) + 1)|I| = \tilde{f}(k)|I|$

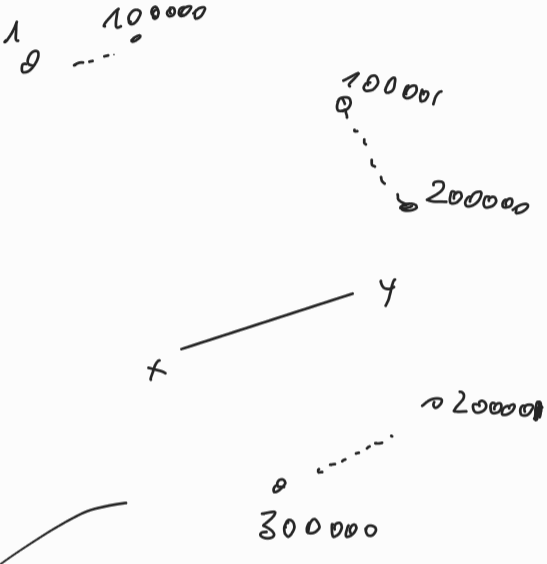
Bsp:



$(G, k=4)$

$\Rightarrow$ Rule 2: lösche a, b, c

$k=3$ $k=2$ $k=1$



Rule 1: lösche deg-0 knoten

$k=1$



$\Rightarrow (G', ?)$ hat G' VC der größe ?

$\rightarrow$ JA!

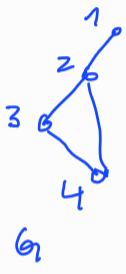
Vertex Cover berührt sogar binären Kern!

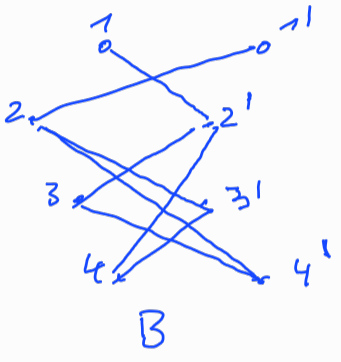
Sei $G = (V, E)$.

Erzeuge disjunkte Kopie $V' = \{v' \mid v \in V\}$ & den bipartiten Graphen:

$$B = (V \cup V', E_B) \quad \text{mit} \quad E_B = \{\{uv'\}, \{v'u\} \mid \{u,v\} \in E\}$$

Bsp:





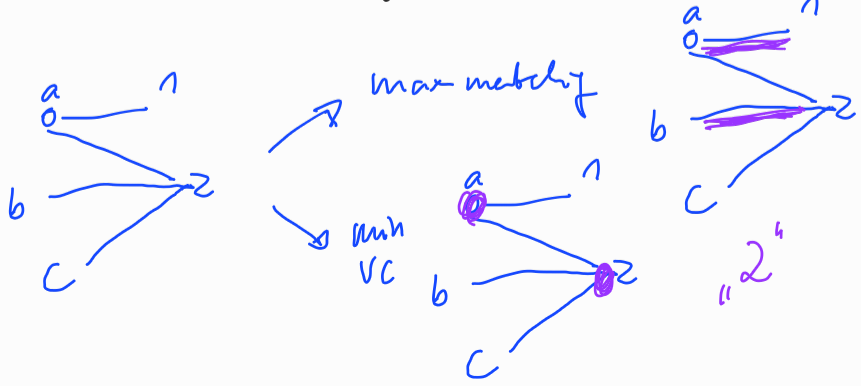
$M \subseteq E$ matching, wenn $e \cap f = \emptyset \ \forall e, f \in M, e \neq f$.

SATZ von König [bzw. Hopcroft-Karb-Alg oder Fluß-alg]

In bipartiten Graphen gilt:

Größe Maximum Matching
= Größe Minimum Vertex Cover

dies kann in poly. Zeit berechnet werden.



Nemhauser Trotter - Thm

Sei $G = (V, E)$ ungw. graph. & B wie oben.
Sei C_B ein min-VC von B

Setze: $C_0 = \{ v \in V : v, v' \in C_B \}$
$V_0 = \{ v \in V :$ genau eines von v, v' liegt in $C_B \}$
$I_0 = \{ v \in V : v, v' \notin C_B \}$

Dann gilt:

(a) Ist D VC von $G[V_0]$, so ist $C_0 \cup D$ ein VC von G

(b) Ein min-VC von G hat die Form $C_0 \cup D$, wobei D ein minVC von $G[V_0]$ ist.

(c) Sei Z = größe min VC von $G[V_0]$, dann gilt: $Z \geq \frac{|V_0|}{2}$

Beweis: Sei C_B minVC von B.
$\Rightarrow$ per Konstruktion von C_0, I_0, V_0 gilt $|C_B| = 2|C_0| + |V_0|$

G:



V_0, I_0, C_0 partition von V

(a) Sei D VC von $G[V_0]$ (rote Dreiecke)

Für VC von G interessieren uns nur noch Kanten zw V_0 & Rest $= C_0 \cup I_0$

& innerhalb $C_0 \cup I_0$

Sei $xy \in E(G)$ bel.

Fall 1: $x \in \overline{I_0}$ (analog $y \in I_0$)

$\Rightarrow$ Def I_0 $x, x' \notin C_B$

B enthält Kanten xy' & $x'y$

& C_B VC von B $\Rightarrow$ $y, y' \in C_B$

Def C_0 $\Rightarrow$ $y \in C_0 \subseteq C_0 \cup D$

Fall 2 $x \in C_0$ oder $y \in C_0$

$\Rightarrow$ Kante überdeckt von Knoten in $C_0 \cup D$

Fall 3 $x, y \in V_0$ $\Rightarrow$ Kante xy vollst in $G[V_0]$

& da D VC von $G[V_0]$

$\Rightarrow$ $x \in D$ oder $y \in D$

insbesondere x oder y in $C_0 \cup D$

$\Rightarrow$ $C_0 \cup D$ VC von G.

(b) Sei S min-VC von G.

sehe

$$S_V = S \cap V_0$$
$$S_C = S \cap C_0$$
$$S_I = S \cap I_0$$

Dann ist $Q = (V \setminus (I_0 \setminus S_I)) \cup S_C'$

ist VC von B

$(S_C' = \{v' \mid v \in S_C\})$

WEIL:

Sei $\{x, y'\} \in E_B$, wenn $x \in Q$ ✓

Ann $x \notin Q \Rightarrow x \in I_0 \setminus S_I \Rightarrow x \in I_0$

$\Rightarrow x \notin S_I = S \cap I_0 \Rightarrow x \notin S$ (*)

Da $x \in I_0 \Rightarrow x, x' \notin C_B$

wegen $\{x, y'\}, \{x', y\} \in E_B$ & da C_B VC von B

$\Rightarrow y, y' \in C_B \Rightarrow y \in C_0$

Zudem gilt $\{x, y\} \in E(b)$

Da S VC von G & $x \notin S \Rightarrow y \in S$

Zusammen erhalten wir:

$$y \in S \cap C_0 = S_C \iff y' \in S_C' \subseteq Q$$

$\Rightarrow$ Q ist VC von B.

aus minimalität von C_B folgt:

$$2|C_0| + |V_0| = |C_B| \leq |Q| \quad \text{I}$$

Def

Da $Q = (V \setminus (I_0 \setminus S_I)) \cup S_C'$

& $V = C_0 \dot\cup V_0 \dot\cup I_0$ & $S_I \subseteq I_0$

gilt $V \setminus (I_0 \setminus S_I)) = C_0 \dot\cup V_0 \dot\cup S_I$

Außerdem $S_C' \subseteq V'$ disj zu V & $|S_C'| = |S_C|$

$\Rightarrow$ $|Q| = |C_0| + |V_0| + |S_I| + |S_C|$ (I)

(I & II) $\Rightarrow$ $|C_0| \leq |S_I| + |S_C|$

nach (a) ist $C_0 \cup S_V$ VC von G

& $|C_0 \cup S_V| \leq |S_I| + |S_C| + |S_V| = |S|$

da S min VC von G

& $C_0 \cup S_V$ VC von G

& $\leq$ $\Rightarrow$ $C_0 \cup S_V$ min VC von G

per Def ist $S_V = S \cap V_0$ VC von $G[V_0]$

Durch ziehen von S_V durch min VC von $G[V_0]$ erhalten wir die Behauptung.

c) Sei D min VC von $G[V_0]$.

Zeigen $Q = C_0 \cup C_0' \cup D \cup D'$ VC von B

Sei $xy' \in E_B \Rightarrow xy \in E(G)$

Fall 1: $x \in C_0$ oder $y \in C_0$ ✓

Fall 2: • $x, y \notin C_0$

es gilt wie in (a) daß, wenn $v \in I_0$
& $vu \in E(G) \Rightarrow u \in V_0$

da $xy \in C_0 \Rightarrow x, y \notin I_0$

da $V = C_0 \dot\cup I_0 \cup V_0 \Rightarrow x, y \in V_0$

Da D VC von $G \Rightarrow x \in D$ or $y \in D$

also $x \in Q$ or $y' \in D' \subseteq Q$.

$\Rightarrow Q$ VC von B

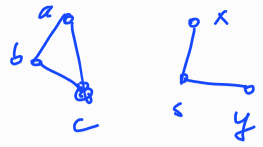
also Q VC von B

$\Rightarrow |C_B| \leq |Q|$

$|C_B| \underset{\text{DEF}}{=} 2|C_0| + |V_0| \leq |Q| = 2|C_0| + 2|D|$

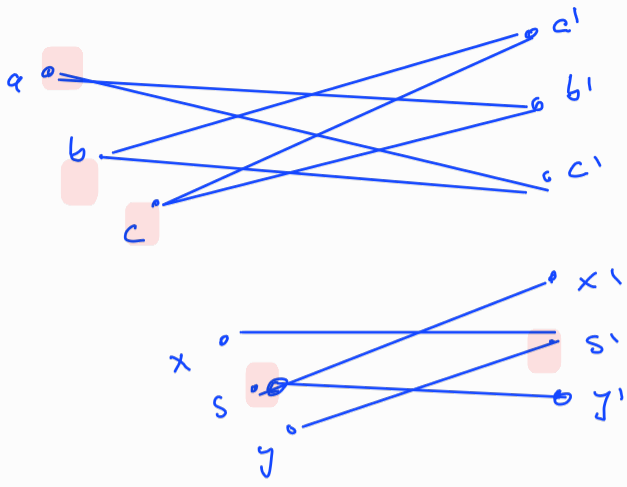
$\Leftrightarrow |V_0| \leq 2|D| = 2z$

$\Rightarrow \frac{|V_0|}{2} \leq z$ ↯

Bsp: $G =$ 

$k = 3$

B:



min VC von B

Bspw: $C_B = \{a, b, c, s, s'\}$

$C_0 = \{s\}$

$V_0 = \{a, b, c\}$

$I_0 = \{x, y\}$

$G' = G[V_0] =$ 

$k' = k - |C_0| = 2$

also kernel $(G', k') = (K_3, 2)$

ist kernel hier JA-instanz: Ja weil $S' = \{a, b\}$ ist VC Größe ≤ 2

$\Rightarrow$ erhalten $S = S' \cup C_0$

Für $k = 2$ $\rightsquigarrow$ kernel $(K_3, 1) \rightarrow$ NEIN

KOROLLAR: VC hat polynom-beschränkten Kernel mit höchstens $2k$ Knoten

Beweis:

VC-Linkernel (G, k)

1. konstruiere B
2. Berechne min VC von B
3. Setze $C_0 = \{v \in V : v, v' \in C_B\}$
   $V_0 = \{v \in V :$ genau einer von v, v' in $C_B\}$
4. $k' \leftarrow k - |C_0|$
5. IF $k' < 0$ OR $|V_0| > 2k$)
   return $(\circ\!\!-\!\!\circ, 0)$ Nein-Inst.
6. return $(G[V_0], k')$

Laufzeit: 1. polyn. zeit
2. — " — Satz von König
3.-6. — " —
insgesamt also polyn. Zeit.

Korrektheit: 1-3 klar. erstmal nur Konstruktionen.
4-5:
$k' = k - |C_0|$, wissen Nem.Trotter (G)
$C_0 \subseteq$ opt VC von G
$\Rightarrow$ wenn $|C_0| > k$
$(\Leftrightarrow k' < 0)$ dann ist
(G, k) Nein-Inst.
& $(\circ\!\!-\!\!\circ, 0)$ ist korrekt
returned.

nach (c) von Nem.Trotter gilt auch,
z = größe opt VC von $G[V_0]$
muß $z \geq \frac{|V_0|}{2}$ gelten.

also $|V_0| \leq 2|z|$
$\Rightarrow$ wenn $|V_0| > 2k$ dann ist
(G,k) Nein-instanz
& $(\circ\!\!-\!\!\circ, 0)$ wird
korrekt returned.

in jedem anderen Fall wird $(G[V_0], k')$
zurückgegeben.
Es gilt nach Nem-Trotter:

$$\tilde{z} = |C_0| + z$$

$\tilde{z}$: größe opt VC G; z: größe opt VC von $G[V_0]$

also (G,k) JA-inst $\Leftrightarrow$ $\tilde{z} \leq k$
$\Leftrightarrow$ $|C_0| + z \leq k$
$\Leftrightarrow$ $z \leq k'$
$\Leftrightarrow$ (G',k') JA-instanz.

Da $|V(G[V_0])| = |V_0| \leq 2k' \leq 2k$

$\Rightarrow$ linear kernel (# knoten linear)

für VC ☐

Auf diesem $O(k)$ Kernel kann man jetzt Brute-Force alle 2^k Teilmengen testen $\Rightarrow$ erhält FPT-Alg.

Folgendes allgemeines Resultat:

Theorem: Sei D Entscheidungsproblem. Dann gilt:

$(D,p) \in FPT \iff$ es gibt Kernelisation-Alg. für (D,p).

Beweis:

„$\Leftarrow$" wenn es Kernel-alg. für (D,p) gibt
so erhalte wir in $O(|I|^c)$ Zeit
eine Instanz $(I', p(I'))$
der größe $|I'| \leq g(p(I)) =: k_I$

Benutze nun bel. Brute Force Algorithmus
mit Laufzeit $f(|I'|) \leq f(k_I)$

$\Rightarrow$ beschränkte Laufzeit: $O(|I|^c + f(k_I)) \leq O(|I|^c \cdot \tilde{f}(k_I))$ also FPT

Kernel berechnen

bel. Alg auf Kernel I'

$\tilde{f}$ Aufl. (setze $\tilde{f}(k) = f(k)+1$)

$\Rightarrow$ Sei: $(I, k_I) \in (D,p)$ & $(D,p) \in FPT$.
$k_I = p(I)$

sei A ein FPT-alg für (D,p).
der jede Instanz (I, k_I) in
Zeit $T_A(|I|, k_I) \leq f(k) \cdot |I|^c$ Zeit entscheidet.

(f berechenbar, i.e. nicht notw. berechenbar in polyn. Zeit!)

Betrachte folgenden Alg.

Kernel (I, k_I)

1. Führe Alg. A auf I für höchstens n^{c+1} Schritte aus.
2. Falls A innerhalb dieser Schritte terminiert:
   Falls A JA liefert: return (konst JA-inst)
   — " — NEIN — " : return (konst NEIN inst.)
3. ELSE return (I, k_I)

Korrektheit: klar, zurückgegebene Instanz immer equivalente Antwort wie (I, k)

Polynomzeit: # Schritte poly of n^{c+1} Schritte begrenzt
⇒ Gesamtlaufzeit poly.

Beweisgröße:

Fall 1: A hält innerhalb von n^{c+1} Schritten

$\Rightarrow$ return mit Konstanter Größe

Fall 2: A hält nicht innerhalb von n^{c+1} Schritten

Dann gilt:

$$n^{c+1} \leq T_A(n,k) \leq f(k_I)\, n^c$$

n^{c+1}: Laufzeit Länge da nicht gehalten nach n^{c+1} Schritten

$\Rightarrow$ $n < f(k_I)$ d.h. die ursprüngliche Instanz hat schon größe die durch $f(k_I)$ beschränkt ist

in beiden Fällen ist zurückgegebene Instanz größe durch $f(k_I)$ beschränkt

□

Proposition: Sei (D, p) param. Problem.

Es gibt einen Alg. der jede Instanz (I, k) in Zeit $f(k) \cdot |I|^c$ entscheidet

$\Leftrightarrow$

Es gibt einen Alg. der jede Instanz (I, k) in Zeit $\tilde{f}(k) + |I|^{\tilde{c}}$ entscheidet

[$c, \tilde{c}$ Konstanten, $f, \tilde{f}$ berechenbar]

Beweis: „$\Leftarrow$" Ann. es gibt $\tilde{f}(k) + |I|^{\tilde{c}}$ - Zeit Alg.

Für jede nichtleere Instanz I gilt $|I| \geq 1$ & daher $\tilde{f}(k) \leq \tilde{f}(k) \cdot |I|^c$

$$\Rightarrow \quad \tilde{f}(k) + |I|^c \leq \underbrace{(\tilde{f}(k) + 1)}_{f(k)} |I|^c \leq f(k) \cdot |I|^c.$$

„$\Rightarrow$" Ann. es gibt $f(k) \cdot |I|^c$ Zeit Alg.

Betrachten 2 Fälle.

Fall 1: $|I| \leq f(k) \Rightarrow f(k) \cdot |I|^c \leq f(k) \cdot f(k)^c = f(k)^{c+1}$

Fall 2: $f(k) < |I| \Rightarrow f(k) \cdot |I|^c \leq |I| \cdot |I|^c = |I|^{c+1}$

Sehe $\tilde{c} = c + 1$. $\Rightarrow f(k) \cdot |I|^c \leq \max\{f(k)^{\tilde{c}}, |I|^{\tilde{c}}\}$

$$\leq \underbrace{f(k)^{\tilde{c}}}_{=: \tilde{f}(k)} + |I|^c$$

□

$\Rightarrow$ bei FPT Alg. macht es keinen Unterschied ob man laufzeit $f(k) \cdot |I|^c$ oder $\tilde{f}(k) + |I|^{\tilde{c}}$ angibt.

k-MAX_SAT

Input: Bool. Formel $\varphi = C_1 \wedge \dots \wedge C_m$ & integer $k \geq 0$

Q: Gibt es Wahrheitsbelegung der Variablen so, sind k Klauseln wahr werden.

Thm: k-MAX SAT ist NP-vollst.

Beweis: $SAT \leq_p$ k-MAX-SAT (siehe $k = m$) ☐

Beobachtung Klauseln C die x & $\bar{x}$ enthalten sind immer erfüllt. Solche Klauseln kann man einfach entfernen & Instanz $(\varphi - C, k-1)$ betrachten.

⇒ um Darstellung zu vereinfachen nehmen wir OBdA an, daß φ keine Klausel enthält mit x & $\bar{x}$ zudem keine leeren Klauseln & keine 2 gleichen Literale in Klausel

Lemma: wenn $k \leq \lceil \frac{m}{2} \rceil \Rightarrow (\varphi, k)$ ist JA-Instanz

Beweis: wähle beliebige Belegung der Variablen $x_1 \dots x_e \to 0/1$

wenn $\geq k$ - Klauseln in φ erfüllt $\Rightarrow$ fertig

wenn $< k$ - Klauseln in φ erfüllt

$\Rightarrow$ $\underbrace{C_1 \dots C_r}_{\text{erfüllt}}$ $\underbrace{C_{r+1} \dots C_m}_{\text{nicht erfüllt,}}$ das sind $> \lceil \frac{m}{2} \rceil$ viele Klauseln

jetzt schaue alternative Belegung von $x_1 \dots x_e$ an, indem man x_i auf 1 setzt wenn vorher $x_i = 0$

bzw x_i auf 0 setzt wenn vorher $x_i = 1$

C_i = FALSE wenn alle Literale in C_i FALSE (C_i = OR-Verknüpfung)

$\Rightarrow$ $\underbrace{C_1 \dots C_r}_{\text{nicht erfüllt}}$ $\underbrace{C_{r+1} \dots C_m}_{\text{erfüllt,}}$ das sind $> \lceil \frac{m}{2} \rceil$ viele Klauseln

$\Rightarrow$ φ hat Wahrheitsbelegung so mind $k \leq \lceil \frac{m}{2} \rceil$ Klauseln wahr. ⊓⊔

$\Rightarrow$ nehmen im Folgenden an, daß $k > \lceil \frac{m}{2} \rceil$ & daher $m < 2k$

D. h. # Klauseln m ist durch Parameter k beschränkt

($O(k^2)$ variables & $O(k^2)$ literals)

Theorem: k-MAX SAT hat Kernel der größe $O(k^2)$.
Dieses Kernel kann in linear gefunden werden.

Beweis: Sei $\varphi = C_1 \dots C_m$, $2k > m$, & keine Klausel C_i enthält x & $\bar{x}$. & keine doppelten Literale

wir können jetzt die Menge $\{C_1, C_2 \dots C_m\} = \mathcal{C}$ der Klauseln partitionieren in

$F_\ell = \{C \in \mathcal{C} : |C| \geq k\}$ lange Klauseln
$F_s = \{C \in \mathcal{C} : |C| < k\}$ kurze (short) Klauseln.

Sei $L := |F_\ell|$

Falls $L \geq k$, dann ist φ JA-Instanz.

warum? jeder $C \in F_\ell$ hat $\geq k$ Literale

nimm erstes $C_1 \in F_\ell$ & setze bel. Literal $\ell' =$ TRUE
$\Rightarrow C_1$ TRUE.

nimm nächstes $C_2 \in F_\ell$ & da $|C_2| \geq k$
existiert Literal $\ell'' \in C_2$
$\ell'' \neq \ell'$
setze jetzt $\ell'' =$ TRUE

⋮

$C_k \in F_\ell$ & da $|C_k| \geq k$ & erst $k-1$ Literale in den vorherigen Schritten auf TRUE
$\Rightarrow$ existiert „unbelegtes" Literal $\tilde{\ell}$ in $C_k \Rightarrow$ setze $\tilde{\ell} =$ true.

$\Rightarrow C_1, \dots C_k$ sind TRUE $\Rightarrow \varphi$ JA Inst.

$\Rightarrow$ bleibt noch Fall $|L| < k$

Betrachte $\varphi_S = \bigwedge_{C \in F_S} C$

Beh: (φ, k) JA-inst. $\Leftrightarrow$ $(\varphi_S, k-L)$ JA-instanz.

Beweis von Beh.

„$\Rightarrow$" wenn (φ, k) JA-inst. ist

$\Rightarrow$ es gibt Belegung sd mind. k Klauseln in φ wahr sind

$\Rightarrow$ höchsten L dieser Klauseln können aus F_C stammen, da $L = |F_C|$

$\overset{|L| \leq k}{\Rightarrow}$ mind $k-L$ Klauseln müssen aus F_S stammen

$\Rightarrow$ $(\varphi_S, k-L)$ JA instanz.

„$\Leftarrow$" Sei $(\varphi_S, k-L)$ JA-inst.

$\Rightarrow$ mind $k-L$ Klauseln aus F_S sind wahr

wähle von diesen wahren Klauseln genau $k-L$ Stück: $C_1 \dots C_{k-L}$

Für C_i wähle $\underline{1}$ Literal daß unter dieser Belegung wahr ist f.a. $i \in \{1 \dots k-L\}$

jedes dieser Literal entspricht Var. x oder $\bar{x}$

Sei X^* die Menge der Variablen die zu den gewählten Literalen gehören.

per Konstruktion $|X^*| \leq k-L$

weise von allen anderen Variablen die nicht in X^* sind, erzwinge die Belegung der Var. in X^* daß alle $C_1 \dots C_{k-L}$ wahr sind.

wir können nun alle anderen Variablen die nicht in $x^{\wedge}$ frei belegen ohne dabei die Eigenschaft, daß $C_1 - C_{k-1}$ wahr sind, zu zerstören!

Betrachte nun $C \in F_\ell \Rightarrow C$ hat $|C| \geq k$ Literale da wir ausschließlich x & $\bar{x}$ gemischt in C

$\Rightarrow$ C kann auf mind k versch. Variablen aufg.

Bisher höchstens $k-L$ Variablen festgelegt.

$\Rightarrow$ in C sind noch mind. $k-(k-L)=L$ Variablen frei.

jetzt nimm usw $C_1 \in F_\ell$

$\Rightarrow$ es gibt mind. L frei Variablen $\Rightarrow$ nimm eine & belege sie sd $C_1 = \text{TRUE}$

$\Rightarrow$ für 2. $C_2 \in F_\ell \Rightarrow$ es gibt $L-1$ frei Variablen

$\Rightarrow$ wähle eine Var. & belege sie sd $C_2 = \text{TRUE}$

usw ...

wir können somit alle L Klauseln $C_1 .. C_\ell \in F_\ell$ auf TRUE setzen

$\Rightarrow$ erhalten $(k-L) + L = k$ Klauseln als TRUE.
(k-L) Klauseln, L Klauseln

$\Rightarrow (\varphi, k)$ ist JA-Inst. – End. Bew. Beh.

bleibt Problem kann große:

φ_S besteht nur aus $m \geq 2k$ Clauseln aus F_S

für jedes $C \in F_S$: $|C| \leq k$

$\Rightarrow$ # Literale in $\varphi_S \leq 2k \cdot k = 2k^2$

$\Rightarrow \varphi_S \in \mathcal{O}(2k^2)$

Reduktion in Linearzeit (einfach φ & C_i säubern).

□

Daraus ergibt sich jetzt folgender FPT-Alg:

UmaxSAT_FPT (φ, k) // φ ohne $x \vee \bar{x}$ in Klauseln oder gleiche Literale in Klauseln oder leere Klauseln.

1. IF $k \leq 0$ return JA
2. Sehe $m = |\varphi|$ = #Klauseln
   IF $k \leq \lceil \frac{m}{2} \rceil$ return JA
3. $F_L = \{C \in \varphi : |C| \geq k\}$
   $F_S = \{C \in \varphi : |C| < k\}$
4. IF $(|F_L| \geq k)$ return JA
5. $k' \leftarrow k - |F_L|$
6. X = Variablenmenge von F_S — kostet wieder das große $O(k^2)$
7. Für jede Wahrheitsbelegung $Bool: X \to \{0,1\}$ — es kommen höchstens $2k^2$ Var. vor $\Rightarrow 2^{2k^2}$ Belegungen
   wenn mind. k' Klauseln aus F_S wahr
   return JA
8. Return NEIN

$\Rightarrow$ Gesamtlaufzeit

$$O(|\varphi| + 2^{2k^2} \cdot \underbrace{O(k^2)}_{\text{prüfe } \leq 0 \text{ Bel. wahr } \varphi_S})$$

$\Rightarrow$ Thm: k-MAXSAT $\in$ FPT.

Bsp: $\varphi = (x \vee y \vee z) \wedge (\bar{x} \vee a \vee b) \wedge (\bar{y} \vee c) \wedge (z \vee \bar{c}) \wedge (\bar{a} \vee b)$

$k = 4$

$|\varphi| = 5 \quad \Rightarrow \left\lceil \frac{m}{2} \right\rceil = \lceil 2.5 \rceil = 3$ (könnum nicht JA returnen, $k \nleq \left\lceil \frac{m}{2} \right\rceil$)

$m = 5 < 2k = 8$

betrachte Klauseln der größe ≥ 4 & < 4

hier F_5 = alle Klauseln von φ.

$\Rightarrow |F_5| = 0$

Variablen x, y, z, a, b, c

$\Rightarrow$ höchstens $2^6 = 64$ mögliche Wahrheitsbelegungen.

eine von diesen ist bspw $x = 1$, $y = 1$, $z = 1$, $a = 0$, $b = 1$, $c = 0$

$\Rightarrow \varphi = \underbrace{(\overset{1}{x} \vee y \vee z)}_{1} \wedge \underbrace{(\overset{0}{\bar{x}} \vee \overset{0}{a} \vee \overset{1}{b})}_{1} \wedge \underbrace{(\bar{y} \vee c)}_{0} \wedge \underbrace{(\overset{1}{z} \vee \bar{c})}_{1} \wedge \underbrace{(\overset{1}{\bar{a}} \vee b)}_{1}$

$\Rightarrow (\varphi, 4)$ JA-instanz.

Zusfg Kernelisierung:

Ziel: große Input auf eine von k-abhäng Größe reduzieren

Input → Reduktion → kernel → Brute Force o.ä.
$O(f(k))$ auf kernel